

Symbiosis Institute of Technology, Nagpur

DEPARTMENT OF COMPUTER SCIENCE • JAVASCRIPT PROGRAMMING LAB

Student: Tanmay Sankulwar

PRN: 24070521058

Course: Semester V (B.Tech)

Experiment No. 7

Student Name: Tanmay Sankulwar

PRN: 24070521058

File Path: EXP7/index.html , EXP7/Casestudy/

Experiment Title

Dynamic Interactive To-Do List Application & Event-Driven Form

Software / Tools Required

1. Visual Studio Code / Antigravity IDE
2. Google Chrome (or modern web browser)
3. HTML5
4. CSS3
5. JavaScript (ES6)

Experiment Program Code

Task 7.a — Dynamic DOM Manipulation To-Do List

File: EXP7/index.html

The application allows users to add tasks dynamically to an unordered list. Each task item includes an inline Edit action (modifying task text via interactive dialog) and a Delete action (removing the node entirely from the DOM tree).

```
<!DOCTYPE html>
<html>
<head>
  <title>To Do List</title>
  <style>
    body {
      font-family: Arial;
      background: #f2f2f2;
      text-align: center;
    }
    .container {
      width: 400px;
      margin: 50px auto;
      background: white;
      padding: 20px;
      border-radius: 8px;
    }
    input {
      padding: 10px;
```

```

        width: 220px;
    }
    button {
        padding: 10px;
        margin: 5px;
        border: none;
        cursor: pointer;
    }
    #addBtn {
        background: green;
        color: white;
    }
    .edit {
        background: orange;
    }
    .delete {
        background: red;
        color: white;
    }
    li {
        list-style: none;
        margin: 10px 0;
        padding: 10px;
        background: #eee;
        display: flex;
        justify-content: space-between;
        align-items: center;
    }
}
</style>
</head>
<body>

<div class="container">
    <h1>To Do List</h1>

    <input type="text" id="taskInput" placeholder="Enter task">
    <button id="addBtn">Add</button>

    <ul id="taskList"></ul>
</div>

<script>
    let input = document.getElementById("taskInput");
    let addBtn = document.getElementById("addBtn");
    let taskList = document.getElementById("taskList");

    addBtn.addEventListener("click", function() {
        let task = input.value;

        if (task == "") {
            alert("Enter a task");
            return;
        }

        let li = document.createElement("li");
        let span = document.createElement("span");
        let editBtn = document.createElement("button");
        let deleteBtn = document.createElement("button");

        span.innerText = task;

        editBtn.innerText = "Edit";
        editBtn.className = "edit";

        deleteBtn.innerText = "Delete";
        deleteBtn.className = "delete";

        editBtn.addEventListener("click", function() {
            let newTask = prompt("Edit task:", span.innerText);
            if (newTask != null && newTask != "") {
                span.innerText = newTask;
            }
        })
    })

```

```
});

deleteBtn.addEventListener("click", function() {
    li.remove();
});

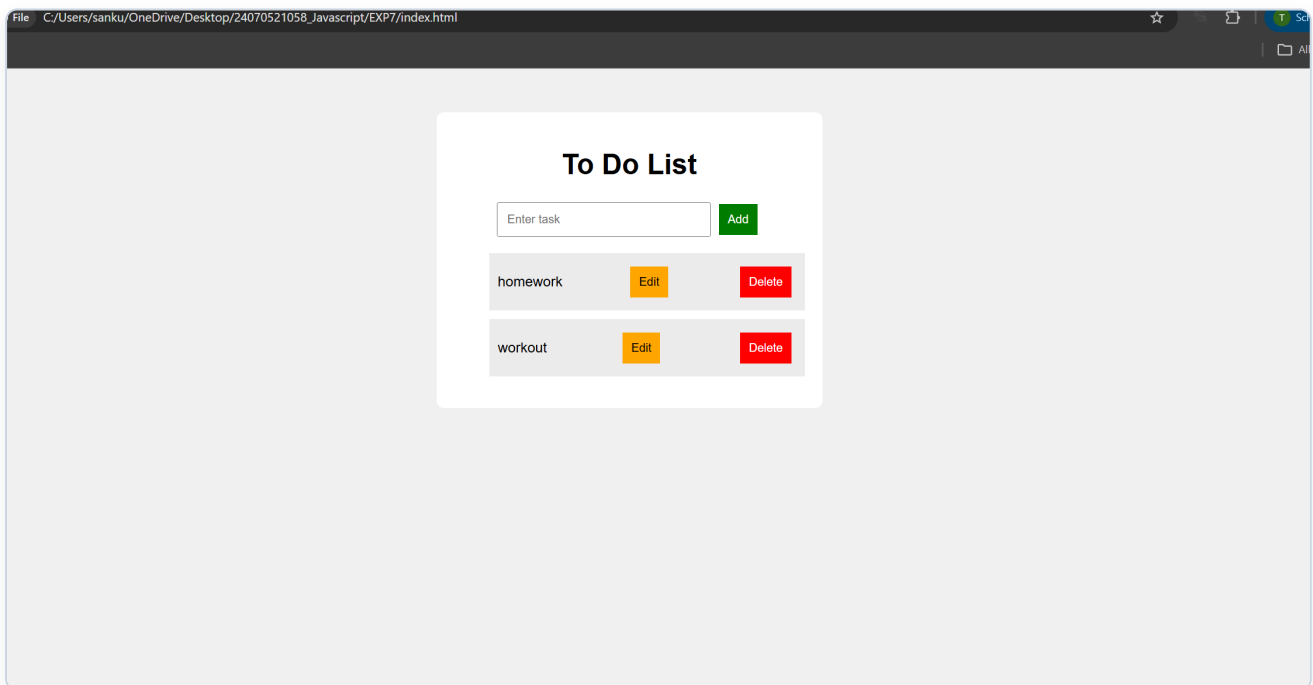
li.appendChild(span);
li.appendChild(editBtn);
li.appendChild(deleteBtn);

taskList.appendChild(li);
input.value = "";
});
</script>
</body>
</html>
```

Output

1. **Task Insertion:** Typing a task and clicking **Add** constructs a new `<li>` element containing the task name and two styled operational buttons (`Edit` in orange, `Delete` in red).
2. **Task Editing:** Clicking **Edit** summons a JavaScript `prompt()` with the current task text prefilled, updating the node's `.innerText` upon confirmation.
3. **Task Deletion:** Clicking **Delete** executes `li.remove()`, instantly purging the item from the live DOM structure.

Screenshot



To Do List Application

Case Study

Case Study — Comprehensive Registration Form with Event Listeners

File: EXP7/Casestudy/index.html , EXP7/Casestudy/script.js , EXP7/Casestudy/style.css

The case study demonstrates a robust registration system driven by multiple standard DOM events: - **Dynamic Select Population:** JavaScript loops automatically populate day options (1–31) and year options (1950–2026). - **focus Event:** Changes input field background color to highlight active user attention (`lightyellow`). - **change Event:** Emits real-time feedback on month selection and checks email pattern validity (`/^[^ ]+@[^ ]+\.[a-z]{2,3}$/`). - **submit Event & Validation:** Prevents default submission (`event.preventDefault()`), validates non-empty inputs, checks password matching, ensures terms checkbox is checked, and sets user focus (`element.focus()`) on any invalid field.

```
// Excerpt from EXP7/Casestudy/script.js
// Dynamic dropdown population
for (let i = 1; i <= 31; i++) {
    let option = document.createElement("option");
    option.value = i;
    option.textContent = i;
    day.appendChild(option);
}

for (let i = 2026; i >= 1950; i--) {
    let option = document.createElement("option");
    option.value = i;
    option.textContent = i;
    year.appendChild(option);
}

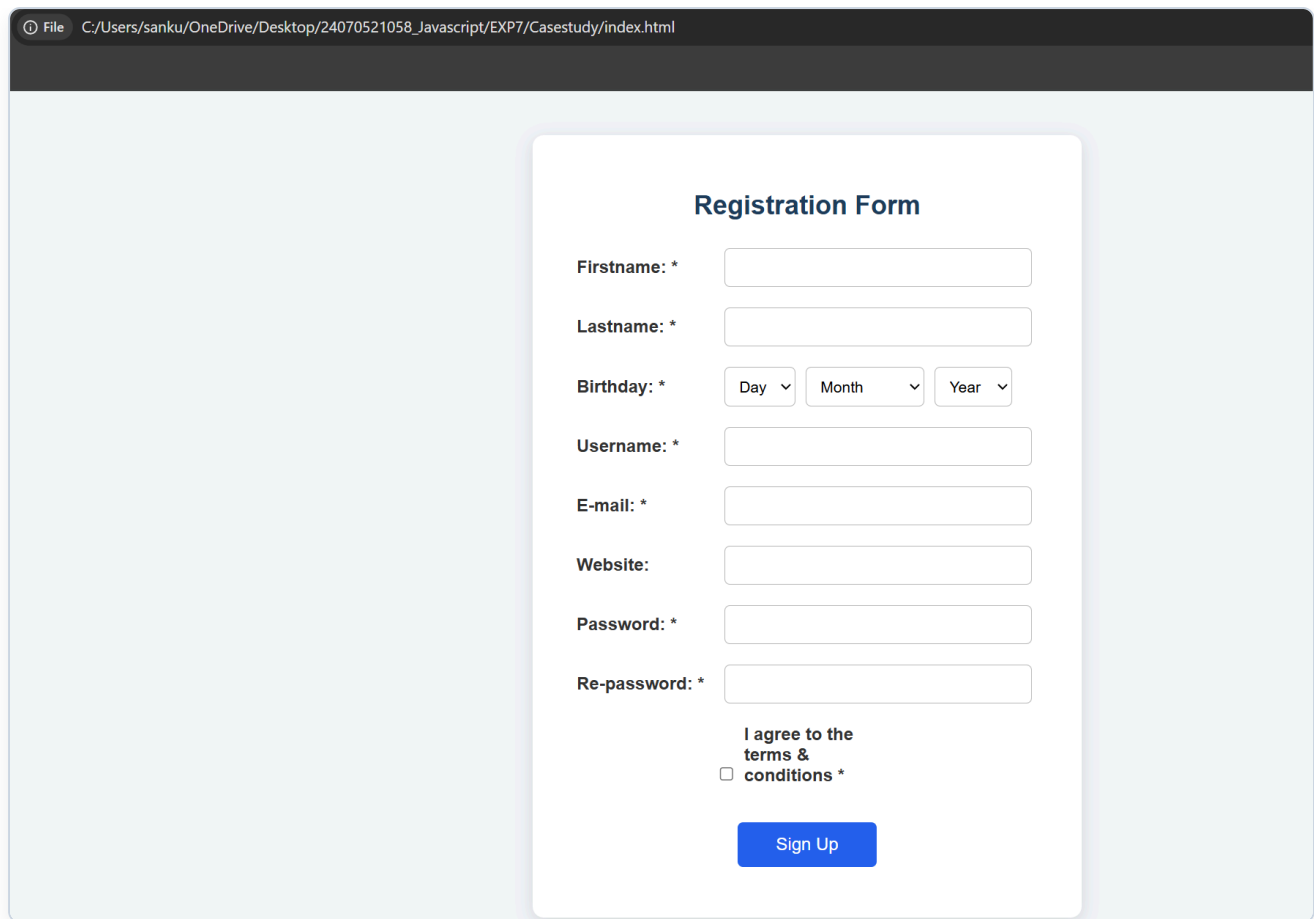
// Event Listeners: focus, change, submit
firstname.addEventListener("focus", function () {
    firstname.style.backgroundColor = "lightyellow";
});

month.addEventListener("change", function () {
    message.textContent = "Month selected: " + month.value;
});

email.addEventListener("change", function () {
    let emailPattern = /^[^ ]+@[^ ]+\.[a-z]{2,3}$/;
    if (!email.value.match(emailPattern)) {
        message.textContent = "Please enter a valid email address.";
    } else {
        message.textContent = "Email is valid.";
    }
});

form.addEventListener("submit", function (event) {
    event.preventDefault();
    // Validate required fields, password match, and terms checkbox
    if (password.value !== repassword.value) {
        alert("Passwords do not match");
        repassword.focus();
        return;
    }
    if (!terms.checked) {
        alert("Please agree to the terms & conditions");
        return;
    }
    message.textContent = "Registration successful!";
});
```

Case Study Screenshot



The screenshot shows a web browser window with the address bar displaying "C:/Users/sanku/OneDrive/Desktop/24070521058_Javascript/EXP7/Casestudy/index.html". The main content area features a light blue background with a white registration form in the center. The form is titled "Registration Form" in bold. It contains the following fields and elements:

- Firstname: *** with a text input field.
- Lastname: *** with a text input field.
- Birthday: *** with three dropdown menus for Day, Month, and Year.
- Username: *** with a text input field.
- E-mail: *** with a text input field.
- Website:** with a text input field.
- Password: *** with a text input field.
- Re-password: *** with a text input field.
- A checkbox labeled **I agree to the terms & conditions ***.
- A blue **Sign Up** button.

Registration Form Case Study

Result / Conclusion

Experiment 7 successfully exhibited dynamic DOM node lifecycles (creation with `document.createElement`, appending with `appendChild`, and destruction with `.remove()`). The case study demonstrated end-to-end event-driven architecture using `addEventListener` for `click`, `focus`, `change`, and `submit` events alongside programmatic dropdown population.